SMART JOB RECRUITMENT AND MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

THOKALA VARSHITHA CHOWDARY (VTU26428)

10211CS224
Full Stack Application Development
Slot : E1-B2

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled " Smart Job Recruitment and Management System " by THOKALA VARSHITHA CHOWDARY (VTU26428) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(THOKALA VARSHITHA CHOWDARY)

Date: 06/05/2026

ABSTRACT

Smart Job Recruitment and Management System is web-based application designed to simplify and streamline the recruitment process for both job seekers and employers. In today's competitive job market, finding suitable employment opportunities and managing recruitment efficiently has become increasingly challenging. This system provides a centralized platform where job seekers(Students) can create profiles, upload resumes, and apply for jobs, while employers can post job openings, review applications, and manage candidate selection processes.The proposed system aims to reduce the gap between recruiters and job seekers by offering features such as job search, application tracking, resume management, and real-time updates on application status. It enhances user experience through a simple and interactive interface, ensuring easy navigation and accessibility.

Keywords: Full-Stack Development, Spring Boot, Java, Thymeleaf, H2, Recruitment System, File Storage, Web Application.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
MVC	Model-View-Controller
RDBMS	Relational Database Management System
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

The recruitment process has shifted from traditional newspaper advertisements to centralized online job portals. The JobPortal application is developed to simplify the hiring process for both employers and job seekers. By leveraging Java Spring Boot, this platform provides a secure, high-performance solution for job posting, talent discovery, and application tracking, ensuring a seamless transition from job search to placement.

1.2 Aim of the Project

The aim of this project is to design and develop a comprehensive recruitment management system that allows students to discover and apply for jobs based on their skills. Simultaneously, it provides administrators with a centralized dashboard to manage job listings, review

student applications, and track recruitment progress in real-time.

1.3 Scope of the Project

The scope of the project encompasses:

- A role-based authentication system for Admin and Students.
- A student-facing portal for searching jobs and applying with resume uploads.
- An admin-facing dashboard for CRUD operations on jobs and application management.
- Secure file storage for managing student resumes.
- An automated email system for application confirmations.
- A persistent relational database to store user data, jobs, and application history.

1.4 Framework

The application is built using the Spring Boot ecosystem, which provides a production-ready environment:

- **Backend:** Spring Boot (Java), Spring Security, Spring Data JPA.
- **Frontend:** Thymeleaf Template Engine, HTML5, CSS3, JavaScript.

- **Database:** H2 Database (for development).
- **Communication:** Spring Mail for SMTP integration.

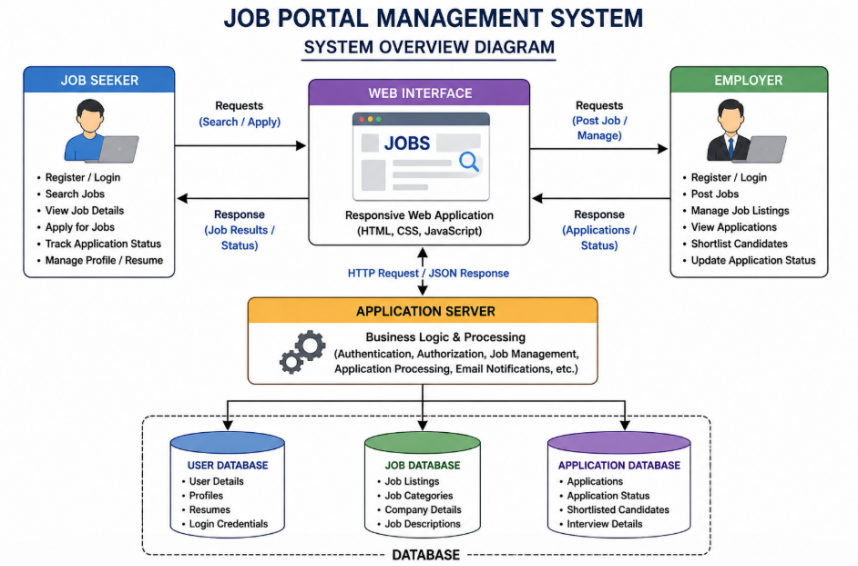

Figure 1.1: System Overview Diagram

Figure 1.1 shows The System Overview Architecture of a Job Portal Management System describes how different components of the system interact to provide a smooth recruitment process. It follows a layered structure that connects users, the application, and the database. The main actors in the system are Students and admin, who access the platform through a web or mobile interface. This interface acts as the entry point where users can register, log in, search for jobs, post job listings, and manage applications.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The job portal market is dominated by global platforms such as LinkedIn, Naukri.com, and Indeed.

- **LinkedIn:** It combines job searching with professional networking. It allows users to build professional profiles, connect with recruiters, and receive job recommendations based on skills and experience. Its networking feature and social proof system make it highly effective for high-level and IT job roles.
- **Naukri.com:** One of the most widely used platforms is Naukri.com, which is known for its large database of resumes and job listings. It is widely used by experienced professionals and companies for mass hiring, with millions of registered users and strong market dominance in India.
- **Indeed:** Indeed is a global job search engine that aggregates job

listings from multiple sources, including company websites and other job portals. It is widely used for its ease of application and comprehensive job database, making it suitable for both freshers and experienced candidates.

The proposed JobPortal differentiates itself by providing a localized, lightweight, and highly secure environment specifically for internal recruitment cycles. It eliminates the noise of social networks and the cost of enterprise subscriptions while maintaining premium security and automated notification features.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend is integrated within the Spring Boot project using the Thymeleaf template engine. This allows for server-side rendering of dynamic HTML. Dependencies include `spring-boot-starter-thymeleaf` and standard web assets (CSS/JS). The UI is styled using modern CSS principles to ensure a responsive and premium user experience across devices.

3.1.2 Structure of the Project

The frontend resources (located in `src/main/resources/templates`) are organized by role:

- **admin-dashboard.html**: The control center for recruiters to post and manage jobs.
- **student-dashboard.html**: The primary interface for students to

browse and apply.

- **register.html & login.html**: Secure forms for user onboarding and authentication.
- **Fragments**: Reusable components like Navbars and Footers to maintain design consistency.

3.1.3 Execution Procedure

To execute the frontend: 1. Ensure the Spring Boot application is running. 2. The templates are automatically served by the Spring Controllers. 3. Access the application via `http://localhost:8080` (or configured port).

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java 17 and Spring Boot 3.3.2. The project uses Maven for dependency management. Critical modules include `spring-boot-starter-security` for robust authentication and `spring-boot-starter-data-jpa` for database abstraction.

3.2.2 Structure of the Project

The backend follows the standard MVC (Model-View-Controller) architecture:

- **Controller Layer**: Handles incoming HTTP requests (AdminController, UserController, AuthController).
- **Service Layer**: Implements business logic (JobService, ApplicationService, EmailService).
- **Repository Layer**: Interfaces with the database using Spring Data JPA.
- **Security Config**: Defines firewall rules and role-based access (SecurityConfig).

3.2.3 Execution Procedure

To execute the backend: 1. Open the project in an IDE (e.g., IntelliJ, VS Code). 2. Run the main class `JobPortalApplication.java` or use `mvn spring-boot:run`. 3. The server will start on the port specified in `application.properties`.

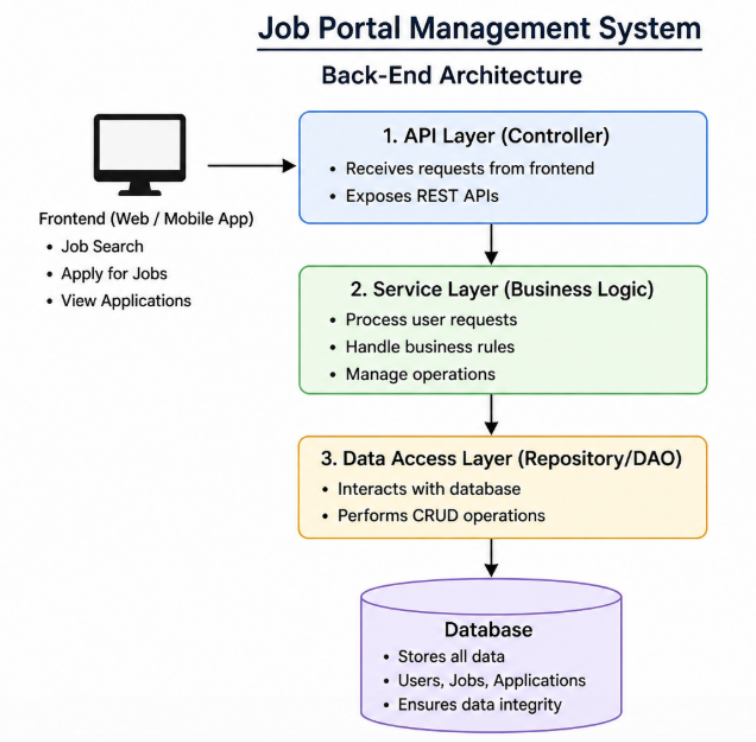

Figure 3.1: Backend Architecture

Figure 3.1 shows that The back-end architecture of a Job Portal Management System defines how the server-side components work together to process user requests, apply business logic, and manage data efficiently. It acts as the core of the system, handling all operations behind the scenes when a user interacts with the frontend. Whenever a job seeker searches for a job or an employer posts a vacancy, the request is sent to the backend, which processes it and returns the appropriate response.

3.3 Database Implementation

3.3.1 Database Configuration

The application utilizes the H2 Database for lightweight development and testing. It is configured as a file-based RDBMS to ensure data persists even after server restarts. The schema is automatically updated by Hibernate (`ddl-auto=update`).

3.3.2 Schema Structure

The database schema is designed with normalized tables to ensure data integrity and efficient querying of job-student relationships.

3.3.3 Tables Created

- **users**: Stores user profiles and roles (ADMIN/STUDENT). Columns: `id`, `username`, `password`, `email`, `role`.
- **jobs**: Stores job listings. Columns: `id`, `title`, `description`, `company`, `location`, `salary`, `posted_date`.
- **applications**: Tracks student applications. Columns: `id`, `user_id`, `job_id`, `resume_path`, `status`, `applied_date`.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a 3-tier MVC architecture. The ”Presentation Tier” (Thymeleaf/CSS) handles the user interaction. The ”Logic Tier” (Spring Boot Services) processes application rules, such as validating resumes or calculating recruitment stats. The ”Data Tier” (JPA/H2) handles the storage and retrieval of relational data.

4.2 DataFlow Diagram

1. A Student logs in and browses the job listings on the User Dashboard.

2. The Student selects a job and uploads a resume to the system.

3. The `FileStorageService` saves the file locally/cloud and returns the file path.

4. The `ApplicationService` creates a record in the database

linking the student to the job.

5. The `EmailService` triggers an automated SMTP mail to the student's registered email.

6. The Admin receives a notification and can view the student's profile and resume from the Admin Dashboard.

4.3 Module 1: User Management Module

This module handles the registration, authentication, and profile management of users in the system. It supports two types of users: Job Seekers and Employers.Job Seekers can create accounts, log in, update personal details, upload resumes, and manage their profiles. Employers can register their organizations, log in securely, and maintain company profiles. Includes features like password encryption, validation, and role-based access control. Ensures secure login and proper session management. It handles all activities related to users in the system. It allows job seekers and employers to register, log in, and manage their profiles securely. Job seekers can upload resumes and update personal details, while employers can maintain company information. This module ensures authentication, authorization, and data security, providing a personalized experience for each user.

4.4 Module 2: Job Management Module

This module is responsible for managing job-related activities within the system.Employers can post new job openings, update job details, and delete job listings. Job Seekers can search for jobs using filters like location, skills, salary, and job type. Displays job descriptions, requirements, and company details. Maintains a structured database of all job postings. It is responsible for handling job-related operations. Employers can post, update, and delete job listings with details such as job title, description, salary, and location. Job seekers can search for jobs using filters like skills and location, and view detailed job information. This module acts as the core component that connects job seekers with available opportunities.

4.5 Module 3: Application and Recruitment Module

This module manages the job application process and recruitment workflow. Job Seekers can apply for jobs by submitting resumes. Employers can view applications, shortlist candidates, and update application status (Accepted/Rejected). Provides application tracking for job seekers. May include notifications (email/SMS) for updates. It manages the complete hiring process. Job seekers can apply for jobs and track the status of their applications. Employers can review ap-

plications, shortlist candidates, and update their status as accepted or rejected. This module improves communication between both parties and ensures an organized and efficient recruitment workflow.

Advantages of this Project

- **Time Efficiency:**The system reduces the time required for job searching and recruitment by automating processes like job posting, application submission, and candidate filtering.
- **Improved Recruitment Process:** Employers can quickly shortlist candidates, review resumes, and manage applications efficiently.
- **Cost Effective:**Reduces the need for traditional recruitment methods such as newspaper advertisements and manual hiring processes.
- **Real-Time Updates:** Users receive instant notifications about job postings, application status, and recruitment decisions.
- **Data Management:** Stores and manages large amounts of user and job data securely, making retrieval and analysis easier.

Disadvantages of this Project

- **Dependence on Internet:** The system requires a stable internet connection, which may limit access for some users.

- **Fake Job Listings:** There is a risk of fraudulent job postings if proper verification mechanisms are not implemented.
- **Technical Issues:** System bugs, server downtime, or software errors can affect user experience and accessibility.
- **Limited Personal Interaction:** Lack of face-to-face communication may reduce the effectiveness of candidate evaluation in early stages.

Chapter 5

RESULTS AND DISCUSSIONS

The Job Portal Management System was successfully designed and developed to provide an efficient platform for both job seekers and employers. The system was tested with various user scenarios, including user registration, job posting, job searching, and application submission.

The results show that job seekers were able to easily create profiles, upload resumes, and apply for jobs without difficulty. Employers were able to post job openings, view applications, and manage candidate selection effectively.

Feature	**LinkedIn**	**Indeed**	**JobPortal**
Internal Control	Low	Moderate	High
Cost for Recruiter	High	Moderate	Zero
Resume Management	Proprietary	Third-Party	Fully Integrated
Target Audience	Global Network	Aggregated Search	Institutional/Private
Security Layer	External	External	Internal (Spring Sec)
Customizability	Low	Low	High
Data Ownership	External	External	Self-Hosted

Table 5.1: Job Portal Feature Comparison

Table 5.1 compares LinkedIn, Indeed, and a custom JobPortal across key features. LinkedIn and Indeed offer broad reach and ease of use but provide limited control, low customization, and rely on external data ownership and security, often with moderate to high costs for recruiters.

5.1 System Screenshots and Results

The Job Portal Application demonstrates a complete workflow from user registration to job application and email notification. The following screenshots illustrate each stage of the system. The system provides a smooth workflow from registration to job application. Users can easily navigate through different modules, and the backend ensures quick processing of requests. Real-time updates and email noti-

fications help keep users informed about their application status.

5.1.1 User Registration Page

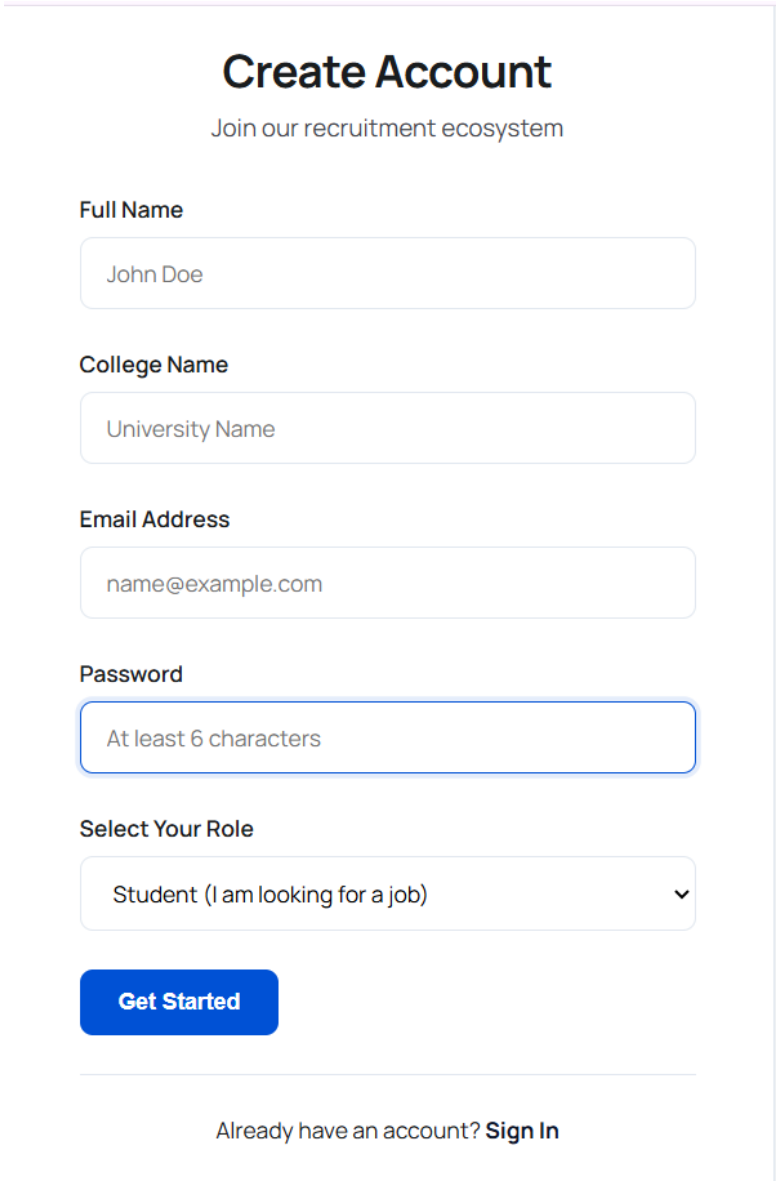

Figure 5.1: User Registration Interface

Figure 5.1 allows new users to create an account by entering personal details and selecting their role as a student. The registration process ensures secure onboarding into the system.

5.1.2 User Login Page

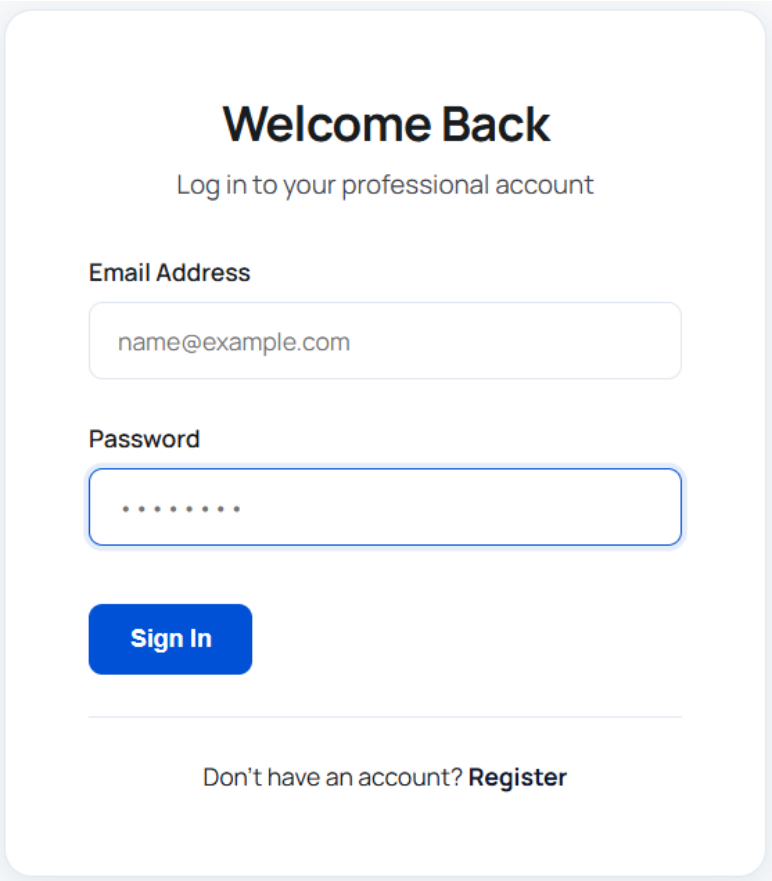

Figure 5.2: Secure Login Page

Figure 5.2 shows Registered users can log in using their email and password. The system validates credentials and redirects users to their respective dashboards.

5.1.3 Job Search and Application

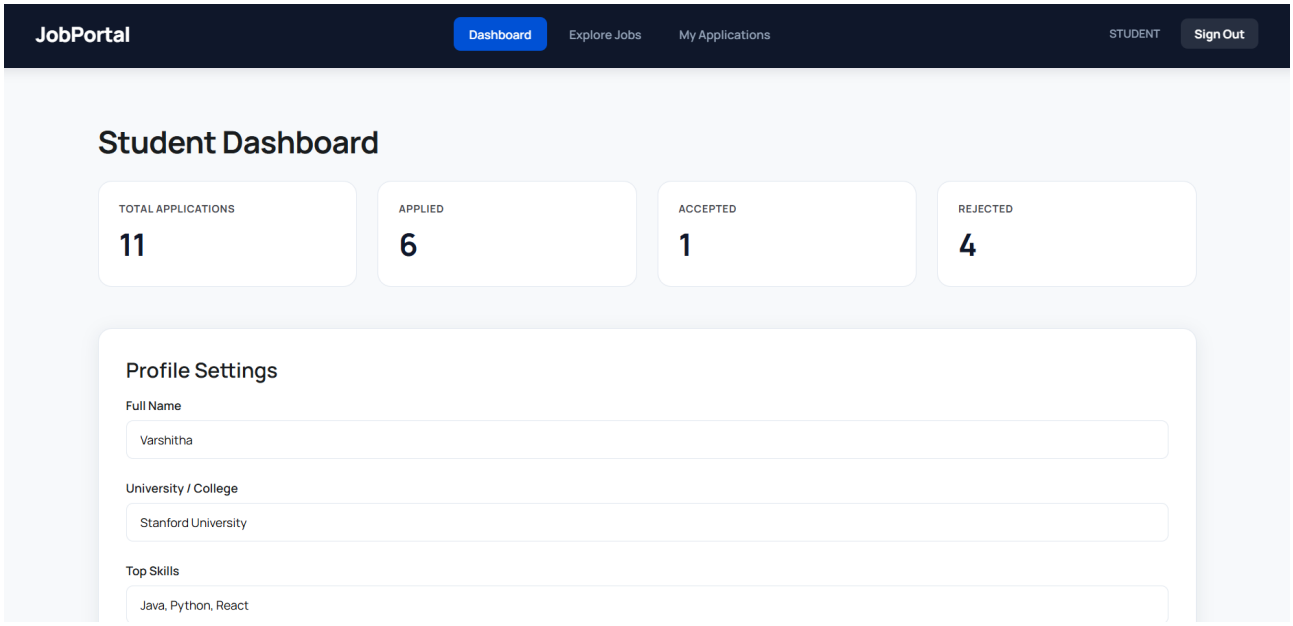

Figure 5.3: Student Job Search and Apply Interface

Figure 5.3 shows Students can browse available job opportunities, filter roles, and apply directly through the portal. Each job listing displays required skills, salary, and location details.

5.1.4 Application Status Tracking

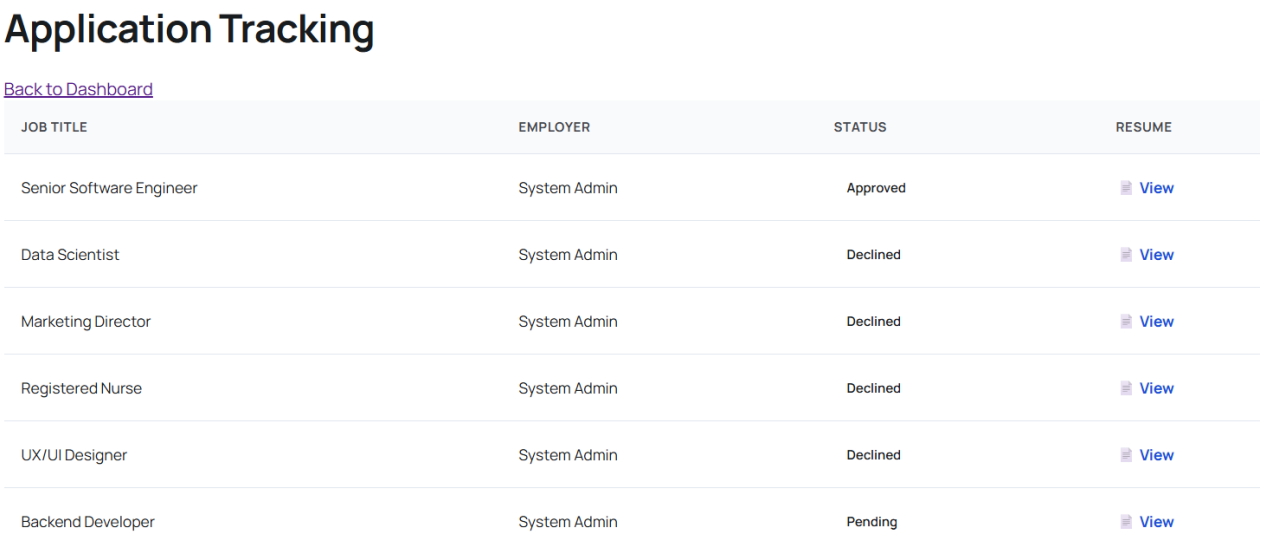

Figure 5.4: Student Application Status Page

Figure 5.4 shows that after applying, students can track the status of their applications such as Applied or Shortlisted.

5.1.5 Admin Dashboard - Application Management

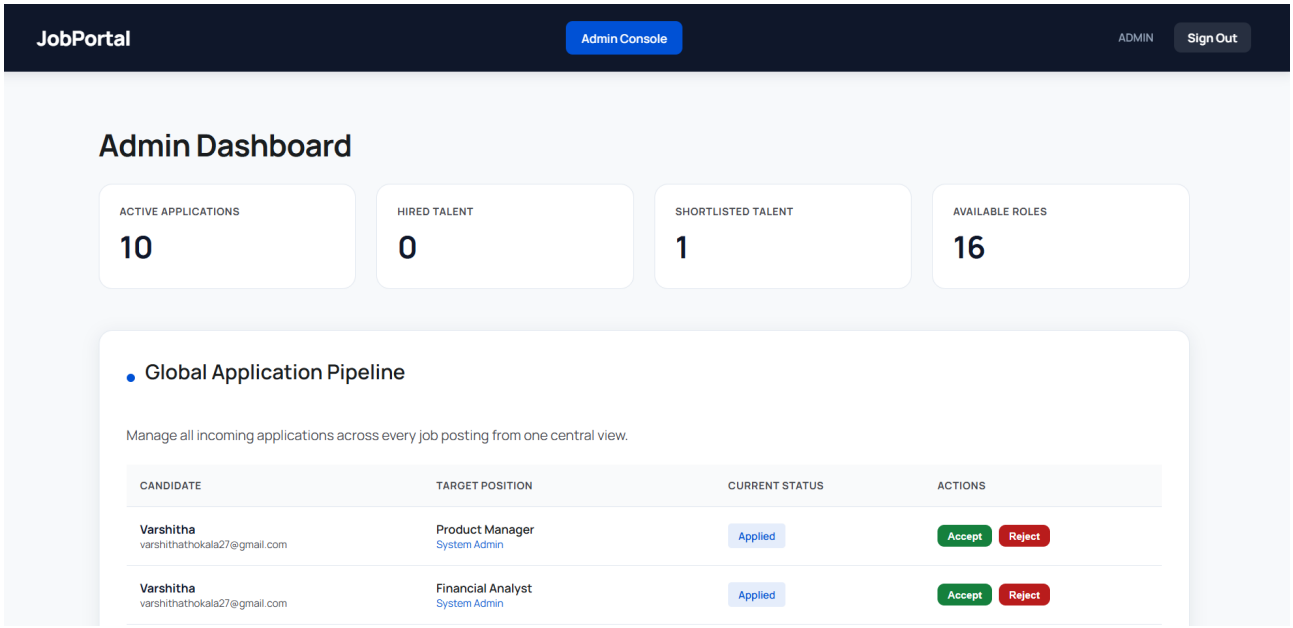

Figure 5.5: Admin Dashboard for Managing Applications

Figure 5.5 shows The admin dashboard allows administrators to view all applications submitted by students. Admins can shortlist or reject candidates based on their profiles.

5.1.6 Email Notification

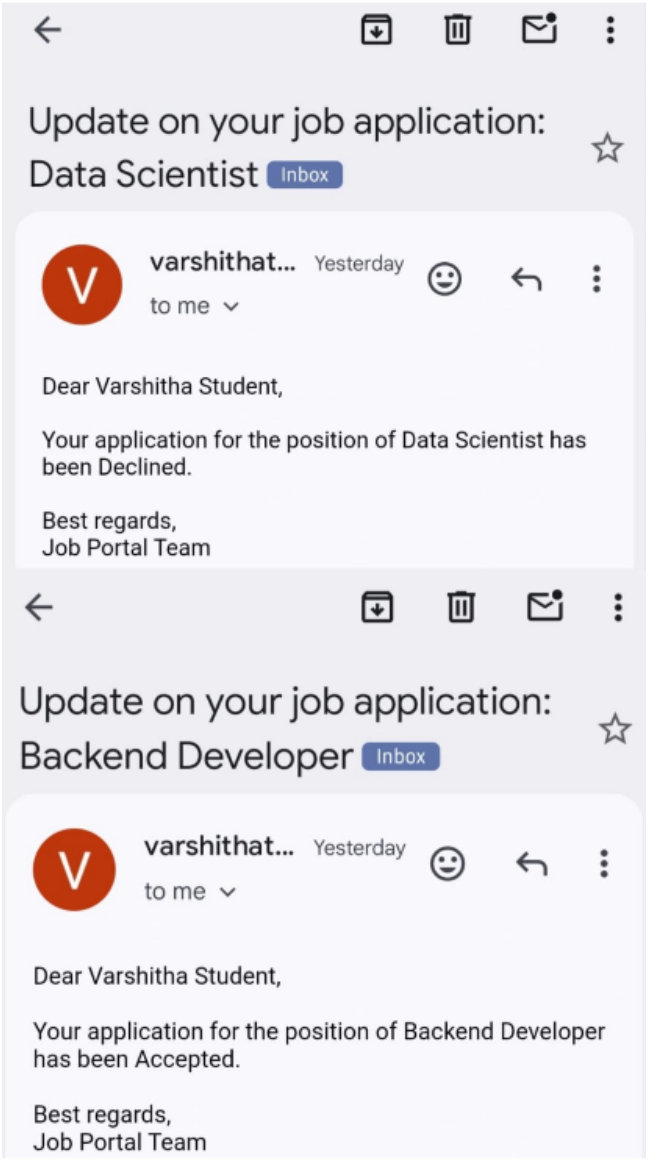

Figure 5.6: Shortlisting Email Notification

Figure 5.6 shows that Once a candidate is shortlisted, an automated email notification is sent to the user. This ensures timely communication regarding application status.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The JobPortal application successfully demonstrates the power of the Spring Boot ecosystem in building enterprise-grade recruitment systems. By integrating secure authentication, automated email notifications, and efficient file storage, the project provides a comprehensive solution for modern hiring needs. The application is highly scalable and provides a solid foundation for any institutional recruitment drive.

6.2 Future Enhancements

- **Cloud Integration:** Migrate file storage to Amazon S3 for better reliability and scalability.
- **Resume Parsing:** Integrate AI/NLP tools to automatically extract

skills from student resumes.

- **Interview Scheduling:** Add a module for scheduling video interviews and sending calendar invites.
- **Advanced Analytics:** Provide admins with graphical insights into application trends and hiring rates.

6.3 SOURCE CODE IMPLEMENTATION

6.3.1 Application Service

```
package com.jobportal.service;

import com.jobportal.entity.ApplicationStatus;
import com.jobportal.entity.Job;
import com.jobportal.entity.JobApplication;
import com.jobportal.entity.User;
import com.jobportal.repository.ApplicationRepository;
import org.springframework.stereotype.Service;

@Service
public class ApplicationService {
    private final ApplicationRepository applicationRepository;
    private final EmailService emailService;
    public ApplicationService(ApplicationRepository applicationRepository,
                              EmailService emailService) {
        this.applicationRepository = applicationRepository;
        this.emailService = emailService;
    }

    public JobApplication apply(User student, Job job) {
        applicationRepository.findByUserAndJob(student, job).ifPresent(a -> {
            throw new IllegalArgumentException("Already applied for this job");
        });
```

```
24          JobApplication app = new JobApplication();
25          app.setUser(student);
26          app.setJob(job);
27          app.setStatus(ApplicationStatus.APPLIED);
28          return applicationRepository.save(app);
29      }
30
31      public JobApplication updateStatus(Long appId, ApplicationStatus status) {
32          JobApplication app = applicationRepository.findById(appId)
33                  .orElseThrow(() -> new IllegalArgumentException("Application not
                        found"));
34          app.setStatus(status);
35          JobApplication saved = applicationRepository.save(app);
36          String to = app.getUser().getEmail();
37
38          String subject = "Job Application Status Updated: " + status.name();
39          String statusMessage = status == ApplicationStatus.ACCEPTED ?
40              "Congratulations! Your application has been ACCEPTED." :
41              (status == ApplicationStatus.REJECTED ?
42              "We regret to inform you that your application has been REJECTED." :
43              "Your application status has been updated to " + status.name());
44          String body = String.format(
45                  "Dear %s,\n\n%s\n\nPosition: %s\n\nBest Regards,\nJob Portal
                        Team",
46                  app.getUser().getName(), statusMessage, app.getJob().getTitle())
                        ;
47
48          emailService.sendEmail(to, subject, body);
49
50          return saved;
51      }
52  }
```

Listing 6.1: ApplicationService.java

6.3.2 User Controller and File Upload Handling

The `UserController` manages the student-facing endpoints of the application. A key feature of this class is the `apply` method, which processes multipart form data to securely save uploaded student resumes to the local file system before registering the job application in the database.

```
package com.jobportal.controller;

import com.jobportal.entity.Job;
import com.jobportal.entity.User;
import com.jobportal.service.ApplicationService;
import com.jobportal.service.JobService;
import com.jobportal.service.UserService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.security.Principal;
@Controller
@RequestMapping("/user")
public class UserController {
    private final UserService userService;
    private final JobService jobService;
    private final ApplicationService applicationService;
    public UserController(UserService userService, JobService jobService,
                          ApplicationService applicationService) {
        this.userService = userService;
        this.jobService = jobService;
        this.applicationService = applicationService;
    }
```

```
29      @GetMapping("/dashboard")
30      public String dashboard(Model model, Principal principal) {
31          User user = userService.findByEmail(principal.getName()).orElseThrow();
32          model.addAttribute("user", user);
33          model.addAttribute("jobs", jobService.getAllJobs());
34          model.addAttribute("applications", applicationService.byStudent(user));
35          return "user/dashboard";
36      }
37      @GetMapping("/jobs/{id}")
38      public String jobDetails(@PathVariable Long id, Model model) {
39          model.addAttribute("job", jobService.getById(id));
40          return "user/job-details";
41      }
42      @PostMapping("/jobs/{jobId}/apply")
43      public String apply(@PathVariable Long jobId,
44                          @RequestParam("details") String details,
45                          @RequestParam("resume") MultipartFile resume,
46                          Principal principal) throws IOException {
47      User user = userService.findByEmail(principal.getName()).orElseThrow();
48          Job job = jobService.getById(jobId);
49          if (!resume.isEmpty()) {
50              Path uploadDir = Paths.get("src/main/resources/static/uploads");
51              if (!Files.exists(uploadDir)) {
52                  Files.createDirectories(uploadDir);
53              }
54              String fileName = System.currentTimeMillis() + "_" + resume.
                    getOriginalFilename();
55              Path filePath = uploadDir.resolve(fileName);
56              Files.write(filePath, resume.getBytes());
57              user.setResumePath("/uploads/" + fileName);
58              user.setExperience(details);
59              userService.save(user);
60          }
61          applicationService.apply(user, job);
62          return "redirect:/user/dashboard?applied=true";
63      }
64  }
```

Listing 6.2: UserController.java

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of Project to Complex Engineering Problem (CEP)

Level	**Attribute**	**Characteristics**	**WKs**	**Justification**
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Uses React.js, Node.js, Express, and MySQL for full-stack development
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and system security
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Includes database design and API integration
WP4	Familiarity	Partially new problems	WK4, WK8	Real-time booking and email features
WP5	Applicable Codes	Limited standard solutions	WK6	Custom ticket booking logic
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Admin and user interaction
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Frontend, backend, and database integration

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of modern technologies	The system uses React.js, Node.js, Express, MySQL, and Nodemailer for full-stack development
EA2	Level of Interaction	Complex module interaction	Handles login, event management, booking, payment simulation, and email notifications
EA3	Innovation	Application of modern techniques	Implements real-time booking updates and automated email confirmation
EA4	Societal Impact	Benefits users	Simplifies event registration and improves accessibility for users
EA5	Familiarity	Beyond standard systems	Combines multiple modules like authentication, booking, and notification into one platform

Table 7.1: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Supports employment and skill development through event participation and management
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation using modern web technologies and automation
SDG 10	Reduced Inequalities	Provides an accessible platform for all users regardless of location
SDG 11	Sustainable Communities	Encourages organized and efficient event management within institutions

Table 7.2: SDG Mapping for the Project

REFERENCES

[1] Naukri.com (2024). Online Recruitment System and Job Listing Services. This platform provides large-scale recruitment solutions, resume databases, and employer hiring tools widely used in India.

[2] LinkedIn (2024). Professional Networking and Intelligent Job Recommendation System. It integrates social networking with recruitment, enabling skill-based hiring and professional connections.

[3] Indeed (2024). Job Aggregation and Search Engine Platform. Indeed collects job listings from multiple sources and provides a centralized interface for job seekers worldwide.

[4] Ian Sommerville (2016). Software Engineering (10th Edition). Pearson Education. This book provides fundamental concepts of system design, architecture, and development methodologies used in software projects.

[5] Oracle (2023). MySQL Documentation and Database Management Concepts. It explains database design, queries, and data handling used in backend development.

[6] Spring (2024). Spring Boot Reference Documentation. This guide provides details on building REST APIs, backend architecture, and enterprise-level applications.

[7] W3Schools (2024). HTML, CSS, JavaScript Tutorials. Used for understanding frontend development and UI design concepts.

[8] GeeksforGeeks (2024). Web Development and Data Structures Resources. Provides coding examples and explanations useful for implementing system logic.

[9] IEEE (2022). Research Papers on Online Recruitment Systems. IEEE publications provide insights into modern recruitment technologies and system improvements.

[10] Google Scholar (2024). Scholarly Articles on Job Portal Systems. Used to review research papers related to job recommendation systems and online hiring platforms.